Login Attempt Monitor — Project Documentation

This document explains the Login Attempt Monitor codebase in detail so you can present it to others. It describes the architecture, runtime flow, important files, how blocking works, where timestamps are handled, how to run and test the project, and suggested demo steps.

Quick Architecture Summary

- Purpose: Watch authentication log lines in real time, detect brute-force attempts, temporarily block offending IPs, persist telemetry, and expose an analyst UI + JSON API.
- Tech: Flask, SQLAlchemy (SQLite), watchdog for file monitoring, simple templated frontend (Tailwind/JS assets included), unit tests with pytest.

Top-level components:
- `app.py` — application factory and startup orchestration
- `config.py` — configuration (env / defaults)
- `database/` — SQLAlchemy DB, models, and migration helpers
- `monitor/` — log parser, brute-force detection, alert creation, file watcher
- `routes/` — UI pages and API blueprint endpoints
- `services/` — authentication helpers, risk scoring, exporting, notifications, statistics
- `utils/` — small helpers, constants, logger
- `templates/` & `static/` — UI

Execution Flow (High Level)

1. `app.py` creates the Flask app using `create_app()`.
2. App config is loaded from `config.py` (env overrides possible via `.env`).
3. Logging is configured and the database is initialized (`database/init_db.py`).
4. Blueprints in `routes/` are registered (UI and API).
5. The `monitoring_service` from `monitor/watcher.py` is configured and started. It:
- Ensures the log file exists
- Reads any initial lines
- Starts a `watchdog.Observer` to watch the file's directory for modifications
6. When the log file changes, new lines are parsed (`monitor/parser.py`) into structured events.
7. Each parsed event is persisted as a `LoginAttempt` and scored via `services/risk_engine.py`.
8. The `monitor.detector` (a `BruteForceDetector`) records attempts and detects repeated failures according to a threshold and time window.
9. When a brute-force is detected, `monitor/alerts.create_alert` is called — it creates an `Alert`, writes a `BlockedIP` (DB) or creates an in-memory runtime block for manual blacklists, and the notification system logs and optionally sends notifications.
10. The UI and API (e.g., `/api/blocked`, `/api/integrations/check-block`) reflect current blocked IPs and alerts. External websites are expected to call the check-block API before processing logins.

Key Data Models (`database/models.py`)

- `LoginAttempt` — stores each parsed login attempt (timestamp, username, ip, source, status, message, risk_score, country).
- `Alert` — stores alerts raised by the detection engine.
- `BlockedIP` — stores temporarily blocked IPs with `blocked_at` and `expires_at`, along with `source` and `reason` fields.

Utility: `utcnow()` returns timezone-aware UTC timestamps used as SQL defaults.

Important Files & What They Do

(Note: file names wrapped in backticks match your workspace.)

- `app.py` — bootstraps the Flask app, initializes DB, registers routes, and starts the monitoring service.
- `config.py` — central configuration object. Important settings:
- `ALERT_THRESHOLD` (fail attempts to trigger alert)
- `ALERT_WINDOW_SECONDS` (time window to consider)
- `BLOCK_DURATION_MINUTES` (how long to block after detection)
- `BLACKLIST_IPS` / `WHITELIST_IPS` (manual lists)
- `LOG_FILE_PATH` (file to watch)

- `database/database.py` — SQLAlchemy `db` instance for ORM usage.
- `database/models.py` — SQLAlchemy models and `to_dict()` helpers used by APIs/UI.
- `database/init_db.py` — helper to create/upgrade the SQLite schema used at app startup.

- `monitor/watcher.py` — the `MonitoringService`:
- Reads new lines from the configured `LOG_FILE_PATH` and persists them.
- Uses `BruteForceDetector` to record attempts and decide when to create alerts.
- When detecting a brute-force, calls `create_alert()` which writes a `BlockedIP` and creates `Alert` objects.

- `monitor/parser.py` — parses raw log lines into a structured dict with keys like `timestamp`, `username`, `ip_address`, `status`, `message`, `source`. It normalizes timestamps to UTC.

- `monitor/detector.py` — contains the `BruteForceDetector` logic: stores short-term history (in memory) keyed by IP/username, evaluates counts within configured time windows and returns a context object when an alert condition is reached.

- `monitor/alerts.py` — coordinates alert creation and persistence. Writes `Alert` and `BlockedIP` rows and logs a notification.

- `routes/*` — UI and API endpoints. Key routes:
- `routes/auth.py` — demo login flow (uses `services.authentication.get_active_block()` to deny blocked IPs before authenticating)
- `routes/api.py` — JSON endpoints for logs, alerts, analytics, blocked IPs, and ingestion endpoints for external websites (`/api/integrations/*`).
- `routes/settings.py` — allows runtime editing of `BLACKLIST_IPS`, `WHITELIST_IPS`, thresholds and durations; updates the `monitoring_service` accordingly.
- `routes/logs.py`, `routes/alerts.py`, `routes/dashboard.py`, `routes/analytics.py` — UI pages for analyst interaction.

- `services/authentication.py` — helper functions used by the demo login page and APIs:
- `get_client_ip(request)` — resolves `X-Forwarded-For` or `remote_addr` and now normalizes IPs via `normalize_ip()` (handles `::1`, IPv4-mapped IPv6, and IP:port forms).
- `get_active_block(ip_address)` — returns either a DB `BlockedIP` record or an in-memory `RuntimeBlock` for manual blacklist entries. It also checks expiry and returns `None` for expired blocks.
- `format_block_time(value)` — returns a user-friendly local timestamp string for UI messages; it was adjusted to show local offsets (e.g., `+02:00`) rather than raw `UTC` only.
- `write_login_event(...)` — appends a log line to `LOG_FILE_PATH` and triggers immediate monitoring ingestion.

- `services/risk_engine.py` — a small scoring routine that computes a risk score for attempts.
- `services/notification.py` — emits notifications and logs alerts (could be extended to email/webhook).
- `services/statistics.py` — composes dashboard summaries and serializes blocked IPs for the UI.
- `services/exporter.py` — CSV export functionality for logs/alerts.

- `utils/helper.py` — small helpers (e.g., CSV parsing for settings, date helpers, country inference stub).
- `utils/logger.py` — logging configuration.

- `templates/` and `static/` — frontend files: pages for login, dashboard, alerts, logs, settings; JS widgets and CSS.

- `tests/` — pytest test suite covering parser, detector, alerting, authentication, and integration API checks.

Blocking / Blacklisting Logic (Detailed)

There are two ways an IP becomes blocked:

1. Automatic blocking from the detector:
- `monitor/detector.py` tracks failed attempts in memory.
- When failure counts exceed `ALERT_THRESHOLD` within `ALERT_WINDOW_SECONDS`, `monitor/alerts.create_alert()` is called.
- `create_alert` writes an `Alert` and creates a `BlockedIP` row in the DB with `blocked_at` and `expires_at` (`BLOCK_DURATION_MINUTES` in `config`).
- The monitoring service's `process_entry()` function prevents any attempts logged from whitelisted IPs from triggering alerts.

2. Manual blacklist (settings):
- `routes/settings.py` allows editing `BLACKLIST_IPS` at runtime. Those are stored in `current_app.config["BLACKLIST_IPS"]`.
- When `get_active_block()` is called and the IP is present in `BLACKLIST_IPS`, a runtime-only `RuntimeBlock` object is returned immediately (no DB row required). This makes the block effective immediately without DB writes.

Normalization caveat (recent fix): IP values from clients may be `::1` (IPv6 loopback) or `::ffff:127.0.0.1` (IPv4-mapped IPv6). To ensure manual blacklists like `127.0.0.1` match these forms, `services/authentication.normalize_ip()` converts these forms to `127.0.0.1` so blacklist checks succeed.

Where blocking is enforced:
- The demo login page (`routes/auth.py`) calls `get_active_block()` and if a block is active, denies the login and flashes a message.
- The API endpoint `/api/integrations/check-block` exposes the active block status to external sites.

Time Handling

- All parsed log timestamps are normalized to timezone-aware UTC datetimes in the parser.
- Database timestamps are stored with timezone awareness (`DateTime(timezone=True)`).
- `format_block_time()` presents times in the server's local timezone with a friendly format like `01 Jul 2026, 03:05:00 PM +02:00` for clarity during demos.

API Endpoints & Usage

Key endpoints (see `routes/api.py`):
- `GET /api/blocked` — list current blocked IPs as JSON.
- `GET /api/integrations/check-block?ip_address=<ip>` — check if an IP is currently blocked (requires `X-API-Key`).
- `POST /api/integrations/login-attempt` — ingest a login attempt from external websites (requires `X-API-Key`).
- Additional endpoints: logs, alerts, analytics, stats used by the dashboard.

Integration pattern for external sites:
1. Call check-block before processing a login.
2. If not blocked, process credentials.
3. POST the login attempt outcome to `/api/integrations/login-attempt`.

Running Locally (how to demo)

1. Create a venv and install dependencies:

```bash
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

2. Start the app:

```bash
python app.py
```

3. Open the UI: `http://127.0.0.1:5000` and use the built-in login page to simulate failures.

4. To see blocking happen:
- Ensure `127.0.0.1` is NOT in `WHITELIST_IPS`.
- Enter invalid credentials repeatedly from the same browser; after `ALERT_THRESHOLD` failures within `ALERT_WINDOW_SECONDS` the system will create an alert and block the IP for `BLOCK_DURATION_MINUTES`.
- You should see a friendly message in the login page with the local expiry time.

5. To test manual blacklist:
- Go to `Settings` in the UI and add `127.0.0.1` to the blacklist and save.
- Attempt to login — it will be blocked immediately and the login attempt will still be written to the log (the app logs blocked attempts for telemetry).

Running Tests

From repository root, with the venv active:

```bash
python -m pytest
```

The repo includes unit tests that validate parser parsing, detector logic, alerts, API integration, and the blacklist behavior.

How to Explain This to Your Teacher (Suggested Talking Points)

1. Start with the problem: centralized detection of brute-force across multiple sites using log ingestion.
2. Explain the flow: ingestion → parsing → scoring → detection → alerting → blocking → UI/API.
3. Walk through `monitor/` to show how file watching and parsing work in near-real time.
4. Show the DB models in `database/models.py` to explain persisted telemetry.
5. Demonstrate blacklisting and how we normalize IP addresses so manual blacklists are reliable.
6. Show `routes/api.py` and explain how other websites can integrate (check-block + post attempts).
7. Run a live demo: trigger a block locally and show the dashboard and alerts.
8. Point to tests as evidence of expected behavior and how to validate the system.

Files Reference (Short)

- `app.py` — app factory and monitoring startup
- `config.py` — environment configuration
- `database/` — contains `models.py`, `database.py`, `init_db.py`
- `monitor/` — `parser.py`, `detector.py`, `alerts.py`, `watcher.py`
- `routes/` — `auth.py`, `api.py`, `dashboard.py`, `settings.py`, etc.
- `services/` — `authentication.py`, `risk_engine.py`, `notification.py`, `statistics.py`, `exporter.py`
- `utils/` — `helper.py`, `logger.py`, `constants.py`
- `templates/` — Jinja2 templates for UI
- `static/` — CSS & JS used by frontend
- `tests/` — pytest test suite

Next Steps (Optional Enhancements You Could Mention)

- Add GeoIP enrichment for alerts (city/country in the UI).
- Add email/webhook alert delivery (extend `services/notification.py`).
- Improve persistence for detector state so restart doesn't lose recent counts (e.g., Redis).
- Add RBAC for the UI.

If you'd like, I can:
- Convert this document into a PDF/slide deck for presenting to your teacher.
- Add a mermaid sequence diagram showing the ingestion and block flow.
- Expand any per-file section into more granular code-line explanations.